

Vision AI 모델 기반 로봇 서비스 구축 - 1

1. 딥러닝 기반 인공지능 모델 학습 기초 이해



[사용자]

↓ (음성)

[Wakeup Word 감지] — “Hello Rokey”

↓

[STT: 음성 → 텍스트] — “ 드라이버 가져와”

↓

[Keyword 추출] — 명령: ‘가져와’, 대상: ‘드라이버’

↓

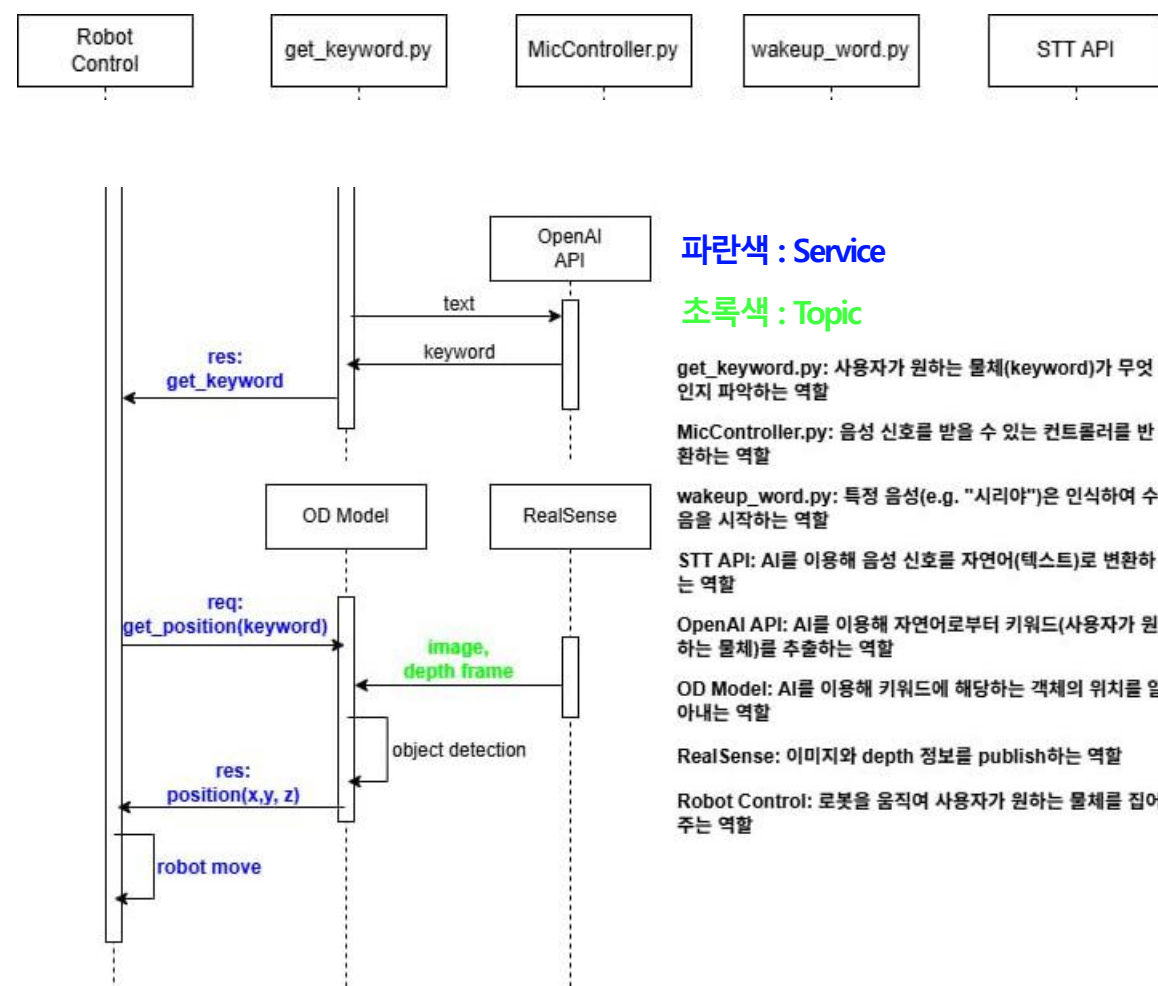
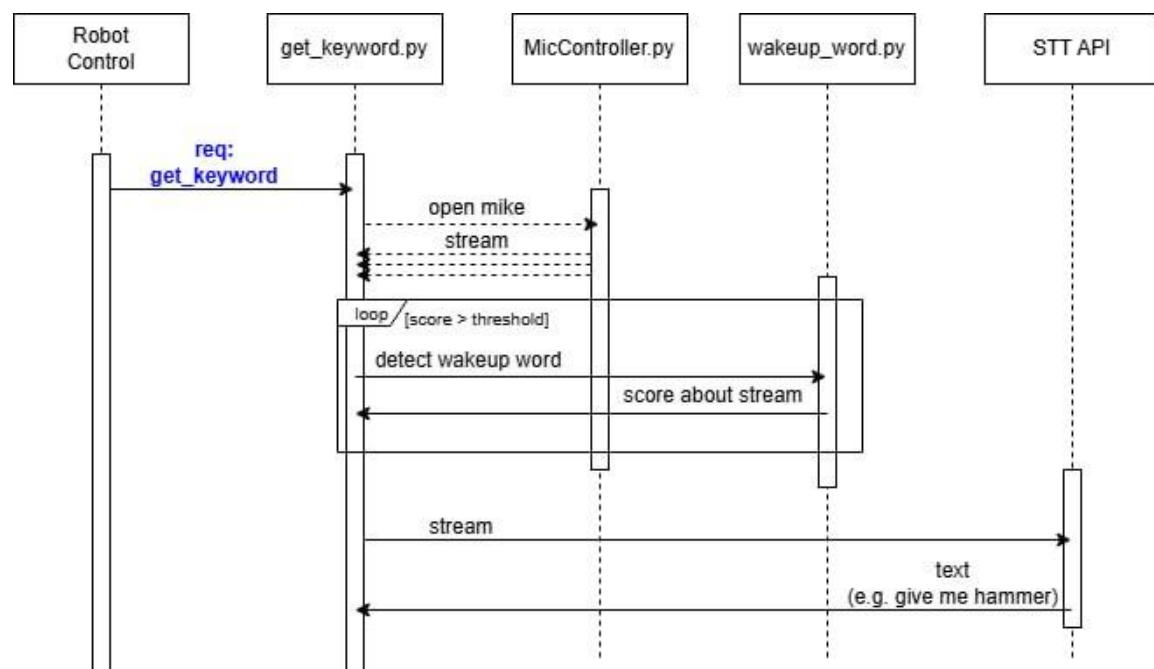
[Object Detection] — YOLO로 '드라이버' 탐지 (좌표 확인)

↓

[로봇 or 시뮬레이터] — 탐지된 객체 위치 이동



기술	역할
Wakeup Word Detection	"로키야" 같은 키워드로 시스템 활성화
STT (Speech-to-Text)	음성 명령을 텍스트로 변환 (예: Whisper, Google STT API)
Keyword Extraction	명령어 해석 ("해머", "오른쪽", "이동" 등 파악)
Object Detection (YOLO)	영상 속 해머 위치 식별
LangChain (옵션)	대화형 제어 흐름, 혹은 multi-tool 의사결정 구조
Movement Controller	ROS2 or Python으로 이동 명령 수행



파란색 : Service

초록색 : Topic

get_keyword.py: 사용자가 원하는 물체(keyword)가 무엇인지 파악하는 역할

MicController.py: 음성 신호를 받을 수 있는 컨트롤러를 반환하는 역할

wakeup_word.py: 특정 음성(e.g. "시리야")을 인식하여 수음을 시작하는 역할

STT API: AI를 이용해 음성 신호를 자연어(텍스트)로 변환하는 역할

OpenAI API: AI를 이용해 자연어로부터 키워드(사용자가 원하는 물체)를 추출하는 역할

OD Model: AI를 이용해 키워드에 해당하는 객체의 위치를 알아내는 역할

RealSense: 이미지와 depth 정보를 publish하는 역할

Robot Control: 로봇을 움직여 사용자가 원하는 물체를 집어주는 역할

▶ 수업 목표

- Object Detection 모델에 대한 라이브러리 및 모듈 설치
- Object Detection 모델을 학습하기 위한 데이터 수집
- Object Detection 모델 학습 및 성능 평가

▶ 세부 목표

- Classification과 Object Detection의 차이
- Bounding Box의 개념
- Confidence, mAP, IoU 소개
- Two-stage vs One-stage
- YOLO 소개
- 음성 인식과 Langchain 기반 LLM 활용
- 강화학습

컴퓨터 비전이 풀어야 할 숙제

- 분류 : 구글 렌즈로 식물을 분류
- 검출 : 물체 위치를 부류 확률(신뢰도)와 함께 직사각형으로 알아냄
- 분할 : 물체가 점유한 영역을 부류 확률과 함께 알아냄
- 추적 : 다중 물체 추적. 움직이는 물체의 위치를 직사각형을 계속 표시
- 행동분석 : 사람 또는 동물이 취하는 행동을 알아냄

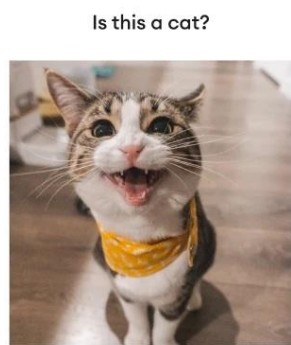
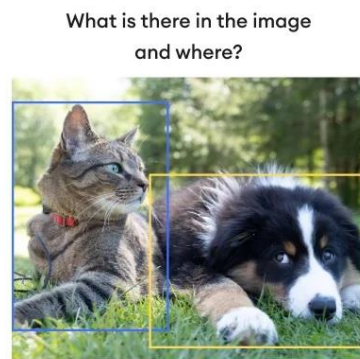


Image Classification



Object Detection

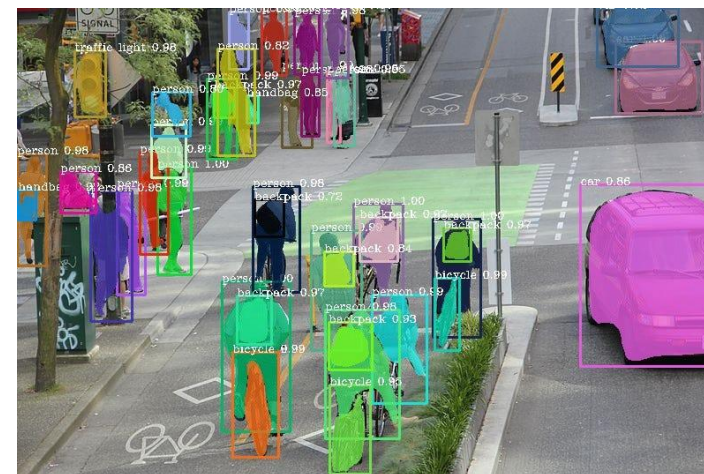


Image Segmentation

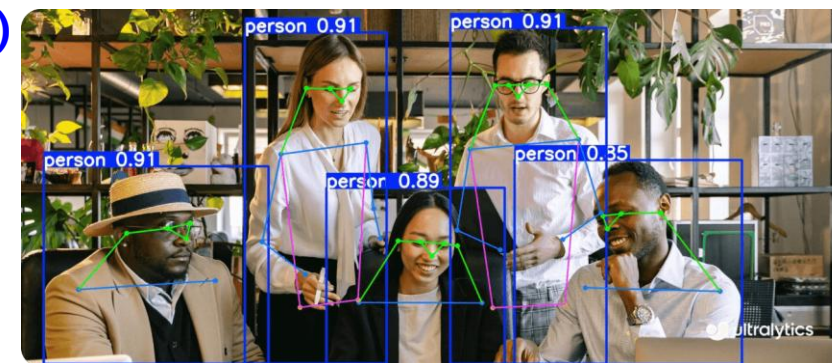
a) Classification



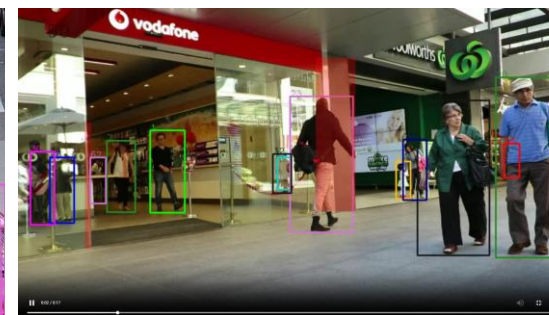
c) CAT



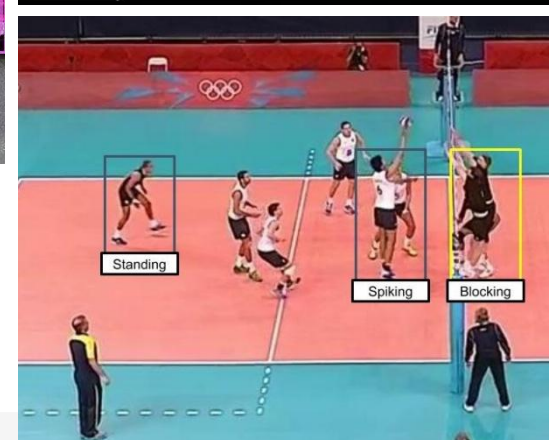
b)



d)



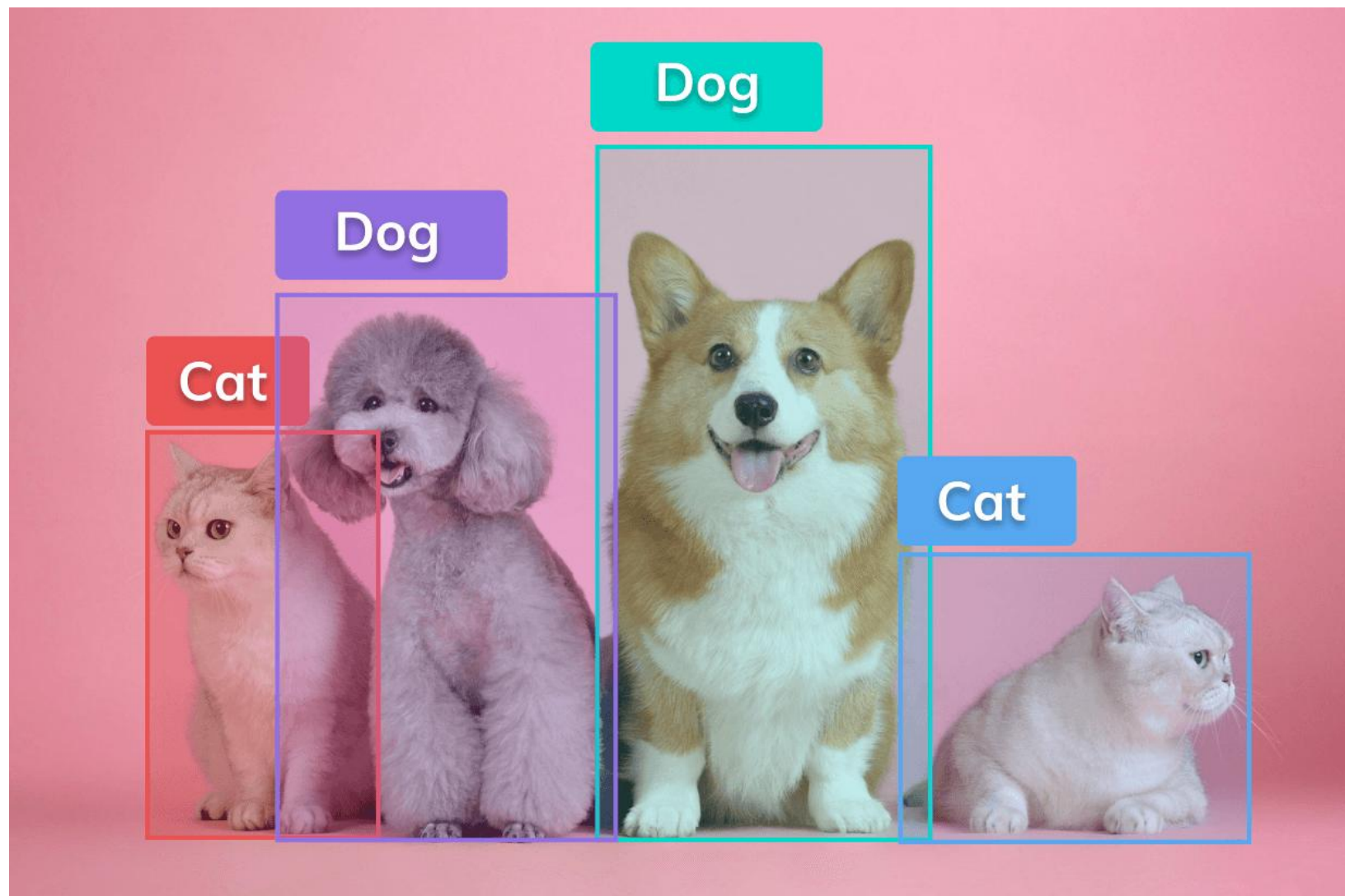
e)





▶ 목적

이미지에서
무엇(대상)이
어디(위치)에 있는지를
모델이 찾게 하는 것



▶ 모델 학습에 필요한 정보

Classification

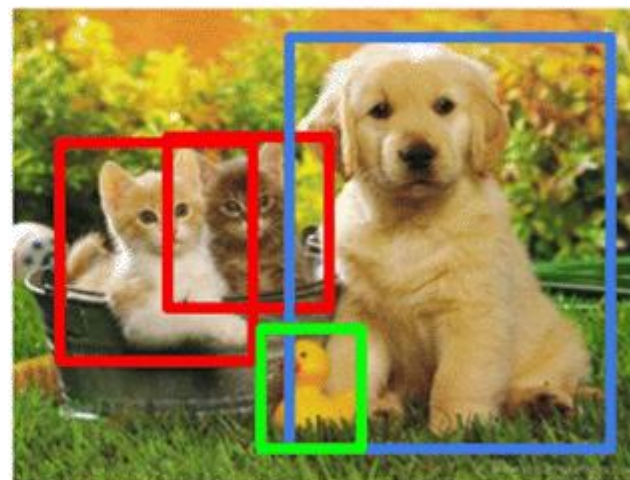
- ① 이미지
- ② 레이블



CAT

Object Detection

- ① 이미지
- ② 레이블
- +
- ③ 위치 정보



CAT, DOG, DUCK

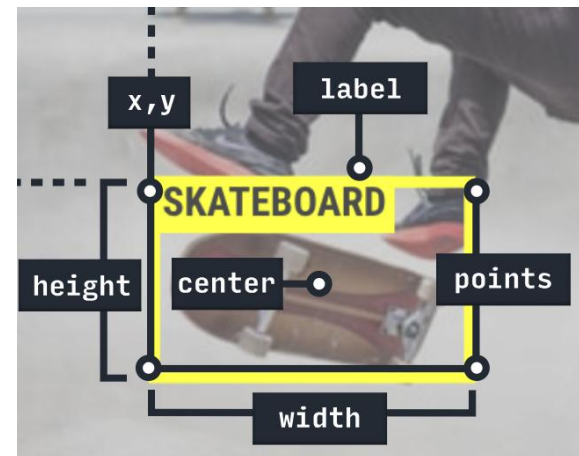
▶ Bounding Box란?

- 이미지 안에서 객체(사물)의 위치를 사각형으로 표시하는 좌표
- 데이터 파일 형식: txt, json, xml 등
- 데이터 형태 [🔗 Xml 데이터 실제 예제](#)
 - (x, y)
 - Width, height
 - (x1, y1, x2, y2)

Bounding box

```
{
  "x": 47.5,
  "y": 171.4,
  "width": 109.1,
  "height": 67.4,
  "points": [[47.5, 171.4], [47.5, 238.8], [156.6, 238.8], [156.6, 171.4]],
  "center": [102.05, 205.1],
  "label": "SKATEBOARD"
}
```

<그림1> Bounding Box 좌표 예시

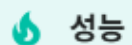


<그림2> Bounding Box 좌표 예시

▶ Yolo8 Model

모델	파일 이름	작업	추론	유효성 검사	교육	내보내기
YOLOv8	yolov8n.pt yolov8s.pt yolov8m.pt yolov8l.pt yolov8x.pt	탐지	✓	✓	✓	✓
YOLOv8-seg	yolov8n-seg.pt yolov8s-seg.pt yolov8m-seg.pt yolov8l-seg.pt yolov8x-seg.pt	인스턴스 세분화	✓	✓	✓	✓
YOLOv8-pose	yolov8n-pose.pt yolov8s-pose.pt yolov8m-pose.pt yolov8l-pose.pt yolov8x-pose.pt yolov8x-pose-p6.pt	포즈/키포인트	✓	✓	✓	✓
YOLOv8-obb	yolov8n-obb.pt yolov8s-obb.pt yolov8m-obb.pt yolov8l-obb.pt yolov8x-obb.pt	방향 탐지	✓	✓	✓	✓
YOLOv8-cls	yolov8n-cls.pt yolov8s-cls.pt yolov8m-cls.pt yolov8l-cls.pt yolov8x-cls.pt	분류	✓	✓	✓	✓

▶ Yolo8 Model



성능

탐지(COCO)

감지(Open Images V7)

세분화(COCO)

분류(이미지넷)

포즈 (COCO)

OBB(DOTAv1)

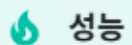
사전 학습된 15개의 클래스가 포함된 [DOTAv1](#)에서 학습된 이러한 모델의 사용 예제는 [지향 탐지 문서](#)를 참조하세요.

모델	크기 (픽셀)	mAPtest 50	속도 CPU ONNX (ms)	속도 A100 TensorRT (ms)	매개변수 (M)	FLOPs (B)
YOLOv8n-obb	1024	78.0	204.77	3.57	3.1	23.3
YOLOv8s-obb	1024	79.5	424.88	4.07	11.4	76.3
YOLOv8m-obb	1024	80.5	763.48	7.61	26.4	208.6
YOLOv8l-obb	1024	80.7	1278.42	11.83	44.5	433.8
YOLOv8x-obb	1024	81.36	1759.10	13.23	69.5	676.7

▶ Yolo11 Model

모델	파일 이름	작업	추론	검증	훈련	내보내기
YOLO11	yolo11n.pt yolo11s.pt yolo11m.pt yolo11l.pt yolo11x.pt	객체 탐지	✓	✓	✓	✓
YOLO11-seg	yolo11n-seg.pt yolo11s-seg.pt yolo11m-seg.pt yolo11l-seg.pt yolo11x-seg.pt	인스턴스 분할	✓	✓	✓	✓
YOLO11-pose	yolo11n-pose.pt yolo11s-pose.pt yolo11m-pose.pt yolo11l-pose.pt yolo11x-pose.pt	포즈/키포인트	✓	✓	✓	✓
YOLO11-obb	yolo11n-obb.pt yolo11s-obb.pt yolo11m-obb.pt yolo11l-obb.pt yolo11x-obb.pt	방향 감지	✓	✓	✓	✓
YOLO11-cls	yolo11n-cls.pt yolo11s-cls.pt yolo11m-cls.pt yolo11l-cls.pt yolo11x-cls.pt	분류	✓	✓	✓	✓

▶ Yolo11 Model



성능

감지 (COCO)

분할 (COCO)

분류 (ImageNet)

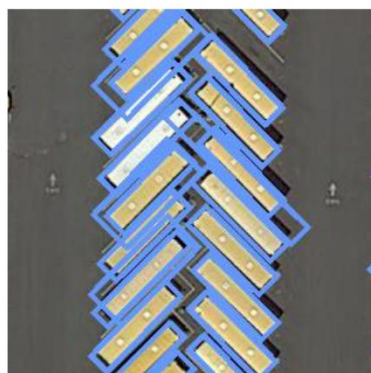
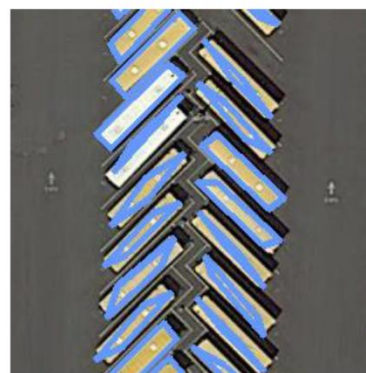
포즈 (COCO)

OBB (DOTAv1)

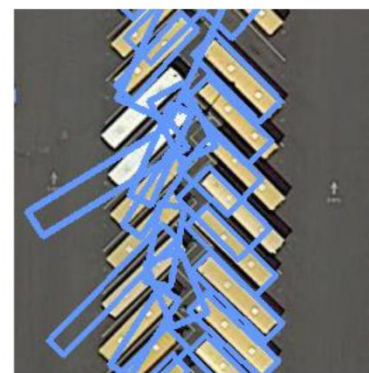
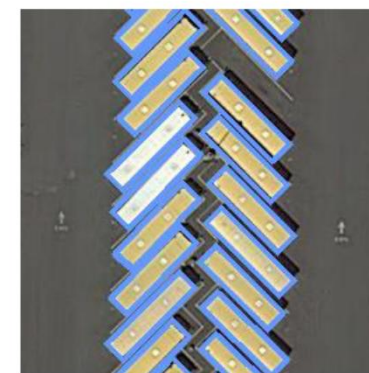
DOTAv1으로 학습된 모델의 사용 예시는 [방향 지정된 객체 감지 문서](#)를 참고하세요. 15개의 사전 학습된 클래스가 포함되어 있습니다.

모델	크기 (픽셀)	mAP ^{테스트} 50	속도 CPU ONNX (ms)	속도 T4 TensorRT10 (ms)	파라미터 (M)	FLOPs (B)
YOLO11n-obb	1024	78.4	117.6 ± 0.8	4.4 ± 0.0	2.7	17.2
YOLO11s-obb	1024	79.5	219.4 ± 4.0	5.1 ± 0.0	9.7	57.5
YOLO11m-obb	1024	80.9	562.8 ± 2.9	10.1 ± 0.4	20.9	183.5
YOLO11l-obb	1024	81.0	712.5 ± 5.0	13.5 ± 0.6	26.2	232.0
YOLO11x-obb	1024	81.3	1408.6 ± 7.7	28.6 ± 1.0	58.8	520.2

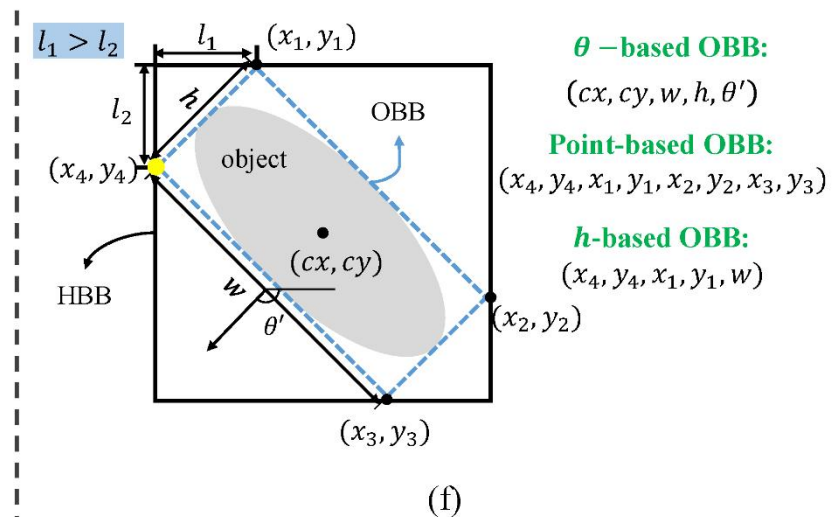
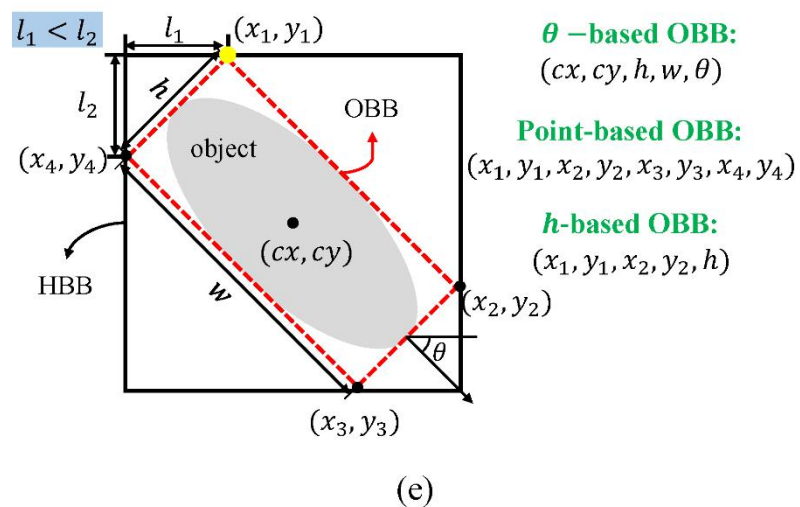
► Oriented Bounding Box란?

(a) θ -based OBB

(b) Point-based OBB

(c) h -based OBB

(d) Mask OBB



▶ Confidence, IoU, mAP의 개념

- Confidence(신뢰도)
→
 ▪ 실제로 존재할 확률
- IoU(Intersection over Union)
→
 ▪ 공식 채점 결과(얼마나 겹치는지?)
- mAP(mean Average Precision)
→
 ▪ 모델의 전체 탐지 성능 종합 평가(정확도와 재현율)

용어	의미	상세 개념
Confidence	객체 존재 + 위치 정확도	"특정 객체가 있을 확률"
IoU(Intersection of Union)	예측 박스와 정답박스 겹침 정도	"예측 박스와 정답박스가 얼마나 겹치는지?"
mAP	모든 Class에 대한 AP의 평균	"전체 평균 성적은 몇 점인가요?" (종합 평가)

► Confusion Matrix

		POSITIVE	NEGATIVE
ACTUAL VALUES	POSITIVE	TP	FN
	NEGATIVE	FP	TN

$$Precision = \frac{TP}{TP + FP}$$

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

$$Recall = \frac{TP}{TP + FN}$$

Confusion Matrix

		Actual		
		1	0	
Predicted	1	TP	FP	Type 1 Error
	0	FN	TN	

Type 2 Error ←

- **Accuracy(정확도)** : 전체 예측 중에서 맞춘 것의 비율
- **Recall(재현률)** : 실제로 존재하는 객체들 중에서 모델이 얼마나 많이 찾아냈는가? (놓치지 않고)
- **Precision(정밀도)** : 모델이 "탐지했다고 예측한 것들 중"에서 실제로 맞는 것의 비율
- **F1 Score** : Precision과 Recall의 조화평균(둘 중 하나라도 낮으면 F1 Score도 낮아짐)

▶ Confusion Matrix

		예측 클래스(Predicted Class)	
		Negative(0)	Positive (1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

오차 행렬을 쉽게 기억하는 방법

1. 왼쪽이 실제, 위가 예측("실제"라고 기억)
2. 실제와 예측이 같으면 T, 다르면 F
3. 음성으로 예측하면 N, 양성으로 예측하면 P

	예측: 정상 (Negative)	예측: 암 (Positive)
실제: 정상 (Negative)	✓ True Negative (TN) → 실제로 정상이고, 정상이라고 잘 예측한 경우	✗ False Positive (FP) → 실제로는 정상인데, 암이라고 잘못 예측한 경우
실제: 암 (Positive)	✗ False Negative (FN) → 실제로 암인데, 정상이라고 잘못 예측한 경우	✓ True Positive (TP) → 실제로 암이고, 암이라고 잘 예측한 경우

- 정확도 (Accuracy):
 - 전체 데이터에서 모델이 얼마나 정확히 예측했는지 나타냅니다.
 - $\text{정확도} = (TP + TN) / (TP + FP + FN + TN)$
- 정밀도 (Precision):
 - 모델이 양성으로 예측한 데이터 중에서 실제로 양성인 비율을 나타냅니다.
 - $\text{정밀도} = TP / (TP + FP)$
- 재현율 (Recall) 또는 민감도 (Sensitivity):
 - 실제 양성 데이터 중에서 모델이 얼마나 잘 찾아냈는지 나타냅니다.
 - $\text{재현율} = TP / (TP + FN)$
- F1-score:
 - 정밀도와 재현율의 조화평균으로, 두 지표가 모두 높은 경우 높은 값을 가집니다.
 - $\text{F1-score} = 2 * (\text{정밀도} * \text{재현율}) / (\text{정밀도} + \text{재현율})$

► Confusion Matrix

1. 정확도(Accuracy)

		예측 클래스(Predicted Class)	
		Negative(0)	Positive (1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

$$\text{정확도} = \frac{\text{예측 결과와 실제값이 동일한 건수}}{\text{전체 데이터 수}} = \frac{(TN + TP)}{(TN + FP + FN + TP)}$$

- 정확도 (Accuracy):
 - 전체 데이터에서 모델이 얼마나 정확히 예측했는지 나타냅니다.
 - 정확도 = $(TP + TN) / (TP + FP + FN + TN)$

2. 정밀도(Precision)

		예측 클래스(Predicted Class)	
		Negative(0)	Positive (1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

$$\text{정밀도} = \frac{\text{True의 비율}}{\text{Positive 전체}} = \frac{TP}{(FP + TP)}$$

- 정밀도 (Precision):
 - 모델이 양성으로 예측한 데이터 중에서 실제로 양성인 비율을 나타냅니다.
 - 정밀도 = $TP / (TP + FP)$

► Confusion Matrix

3. 재현율(Recall)

		예측 클래스(Predicted Class)	
		Negative(0)	Positive (1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

$$\text{재현율} = \frac{\text{예측 결과와 실제값이 동일한 건수}}{\text{실제 양성 데이터 수}} = \frac{TP}{(FN + TP)}$$

- 재현율 (Recall) 또는 민감도 (Sensitivity):
 - 실제 양성 데이터 중에서 모델이 얼마나 잘 찾아냈는지 나타냅니다.
 - 재현율 = $TP / (TP + FN)$

4. F1 Score : 정밀도와 재현율 결합한 평가지표

		예측 클래스(Predicted Class)	
		Negative(0)	Positive (1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

$$\text{F1 Score} = 2 \times \frac{\text{정밀도} \times \text{재현율}}{\text{정밀도} + \text{재현율}}$$

- F1-score:
 - 정밀도와 재현율의 조화평균으로, 두 지표가 모두 높은 경우 높은 값을 가집니다.
 - $F1\text{-score} = 2 * (\text{정밀도} * \text{재현율}) / (\text{정밀도} + \text{재현율})$

► Confusion Matrix

5. ROC, AUC(Receiver Operating Characteristic Curve, Area Under the Curve)

- 다양한 임계값에서의 TPR, FPR을 그린 곡선
- 양성과 음성을 얼마나 잘 구분하는지 시각화한 곡선

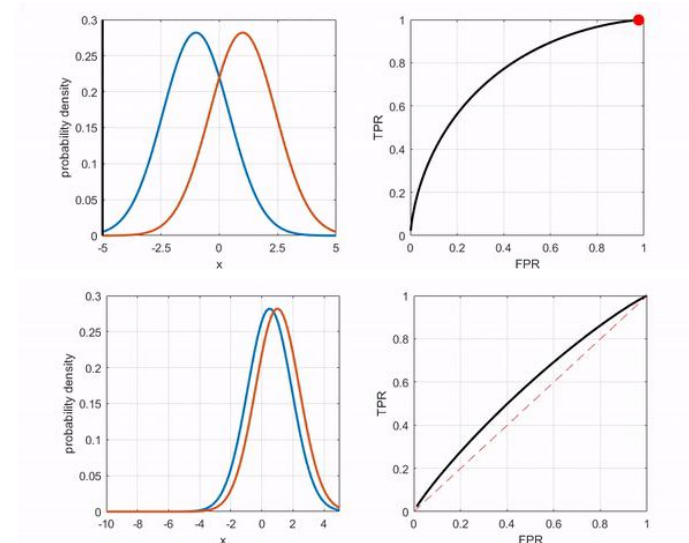
	예측 클래스(Predicted Class)	
	Negative(0)	Positive (1)
실제 클래스 (Actual Class)	Negative(0) TN (True Negative)	FP (False Positive)
	FN (False Negative)	TP (True Positive)
	Positive(1)	

True Positive Ratio(실제 양성을 얼마나 정확히 분류했는지)

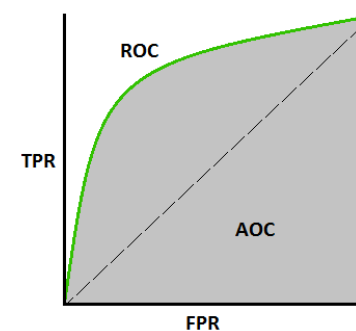
$$\text{TPR} = \frac{\text{TP}}{(\text{TP} + \text{FN})} = \text{Recall}$$

False Positive Ratio(실제 음성중 잘못 분류한 비율)

$$\text{FPR} = \frac{\text{FP}}{(\text{FP} + \text{TN})}$$

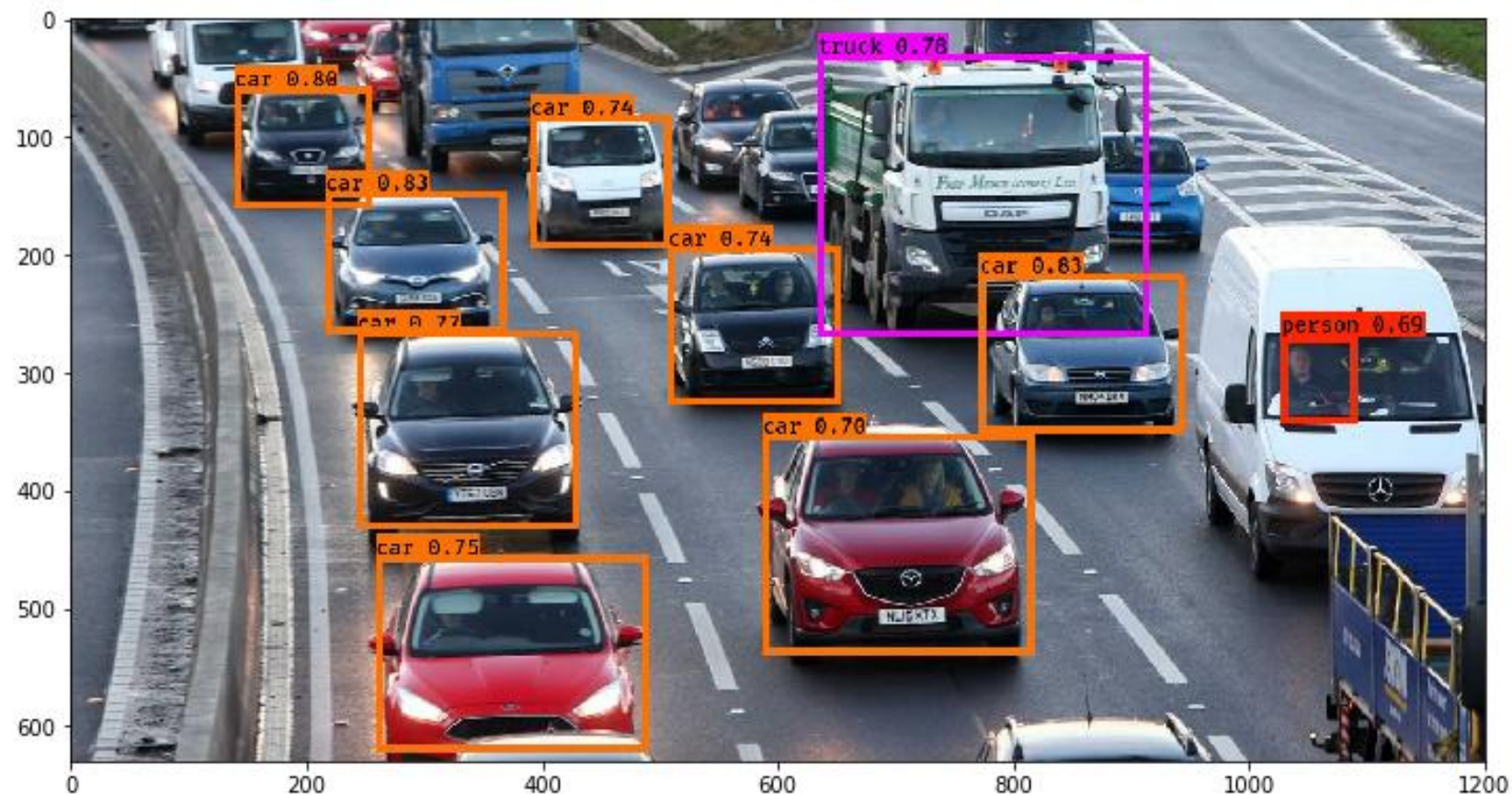


- AUC(면적)는 0.0 ~ 1.0까지의 숫자

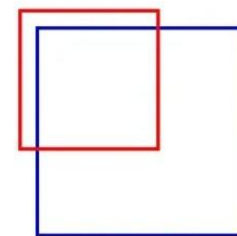
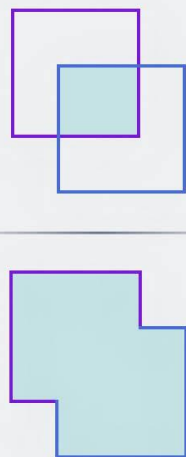


Confidence(신뢰도, 확률 점수) | 학생 입장

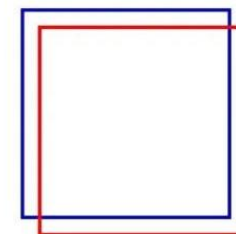
- 이 박스 안에 자동차가 있다고 얼마나 확신하나요?
- 0.0 ~ 1.0 사이(0~100% 의미)



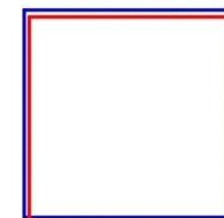
$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}} =$$



Poor



Good



Excellent

01 mAP(mean Average Precision) | 공식 채점 결과(전체 종합 평균)

- 전체 클래스에 대해 얼마나 잘 예측했는지에 대한 종합 점수
- Precision = 정확도
- AP(Average Precision) : 한 클래스에 대한 다양한 임계값에서의 precision을 평균낸 것
- mAP = 모든 클래스에 대한 AP의 평균값

VOC 2007 test	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv	mAP
R-CNN T-Net	64.2	69.7	50.0	41.9	32.0	62.6	71.0	60.7	32.7	58.5	46.5	56.1	60.6	66.8	54.2	31.5	52.8	48.9	57.9	64.7	54.2
R-CNN T-Net BB	68.1	72.8	56.8	43.0	36.8	66.3	74.2	67.6	34.4	63.5	54.5	61.2	69.1	68.6	58.7	33.4	62.9	51.1	62.5	64.8	58.5
R-CNN O-Net	71.6	73.5	58.1	42.2	39.4	70.7	76.0	74.5	38.7	71.0	56.9	74.5	67.9	69.6	59.3	35.7	62.1	64.0	66.5	71.2	62.2
R-CNN O-Net BB	73.4	77.0	63.4	45.4	44.6	75.1	78.1	79.8	40.5	73.7	62.2	79.4	78.1	73.1	64.2	35.6	66.8	67.2	70.4	71.1	66.0

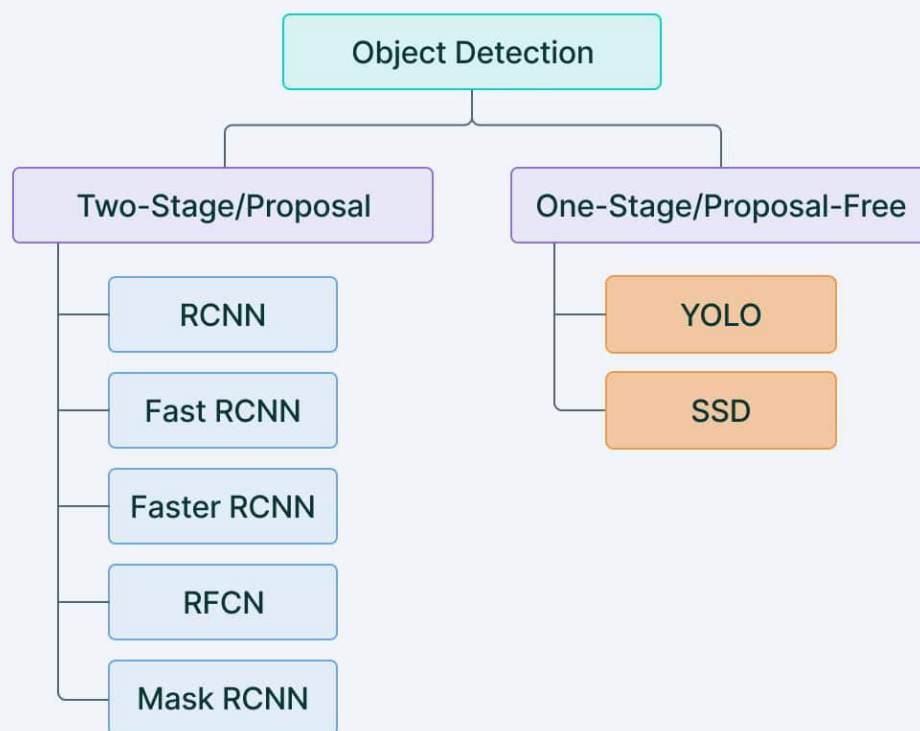
<표 1> R-CNN performance table

지표	의미	점수 범위	언제 사용?
IoU	예측 박스와 실제 박스의 겹침 정도	0 ~ 1	박스 하나하나의 정확도
mAP	전체 클래스 예측의 평균 정확도	0 ~ 1 (or 0~100%)	전체 성능 평가 (대회 기준)

모델의 성능

- 데이터의 다양성
- 데이터의 양
- 클래스간 균형

One and two stage detectors



V7 Labs

▶ Two-stage(Faster R-CNN)

- 두 번 생각해서 정확하게 말하기

“여기 뭔가 있는 거 같은데, 그게 뭔지 확인해볼게요.”

▶ One-stage(yolo)

- 한 번에 직감적으로 툭 말하기

“이건 고양이입니다!”

▶ Faster R-CNN vs YOLO

항목	Two-Stage (Faster R-CNN)	One-Stage (YOLO, SSD)
처리 방식	후보 영역 → 분류	바로 위치 + 분류
Stage1	Regional Proposal(후보생성 → 수백개의 bounding box)	이미지를 한 번에 CNN에 통과시켜 각 Grid셀에서 객체의 존재여부 판단
Stage2	Classification & Regression(각 영역 분류 + box 보정)	Class + BBox + Confidence 동시 예측
속도	느림 (정확한 대신 느림)	빠름 (실시간 가능)
정확도	높음	보통~높음
구조 복잡도	높음	낮음
용도	의료, 위성, 정밀 분석	로봇, CCTV, 드론, 실시간 탐지
특징	정확도는 우수하나 연산량 많아서 느림	작은 객체나 밀집된 객체 탐지 정확도는 떨어질 수 있음 정확도는 차차 개선되고 있음

▶ “어디에 물체가 있는지 먼저 찾아서, 그 다음에 그 위치에 있는 게 뭔지 분류하자!”

구성 단계

- ① CNN으로 이미지에서 특징 추출 (feature map)
- ② RPN (Region Proposal Network) → 후보 영역들 생성 (물체 있을 법한 박스들)
- ③ RoI Pooling → 후보 영역을 고정된 크기로 자름
- ④ Classifier → 각 박스를 물체 클래스 별로 분류

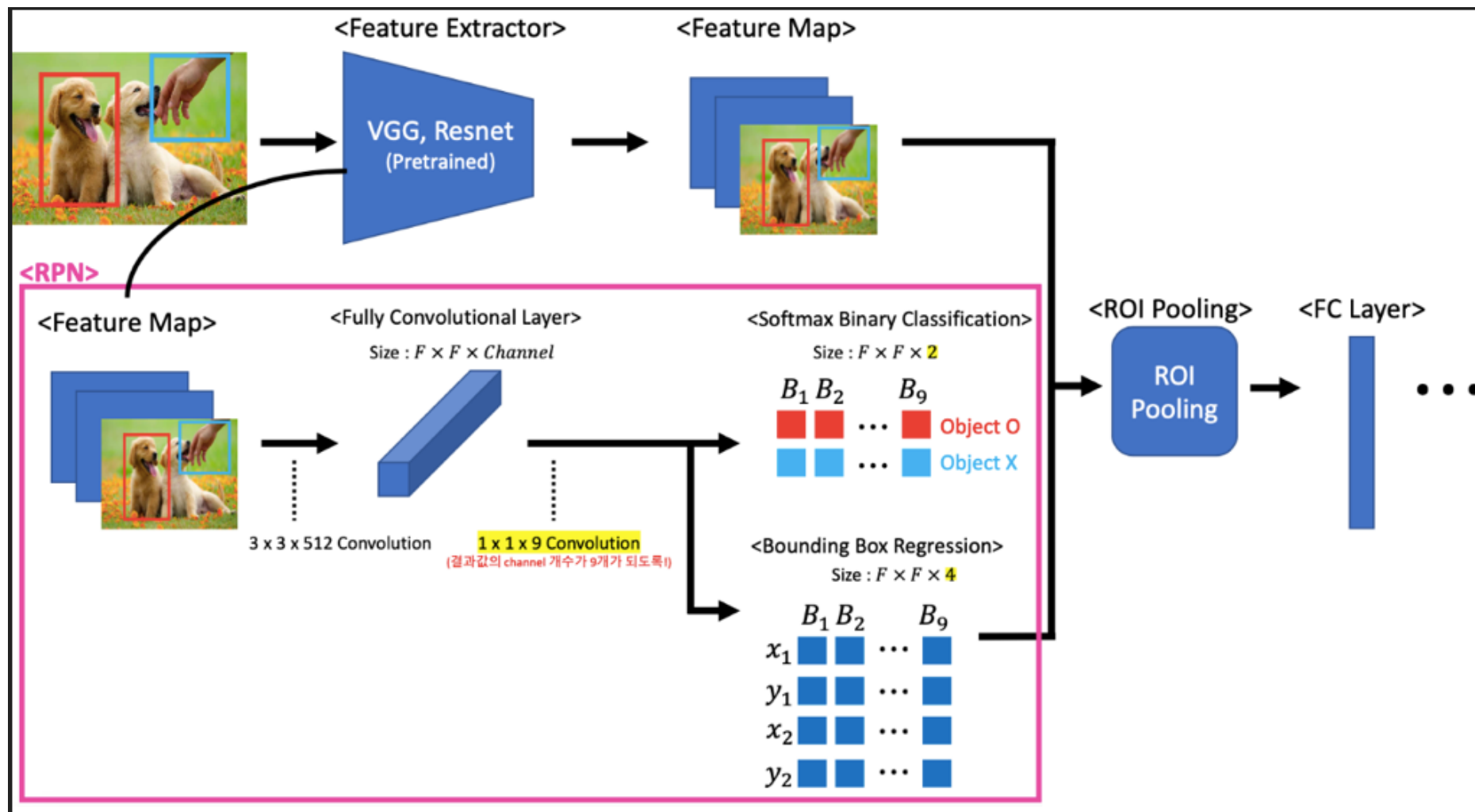
◆ 장점

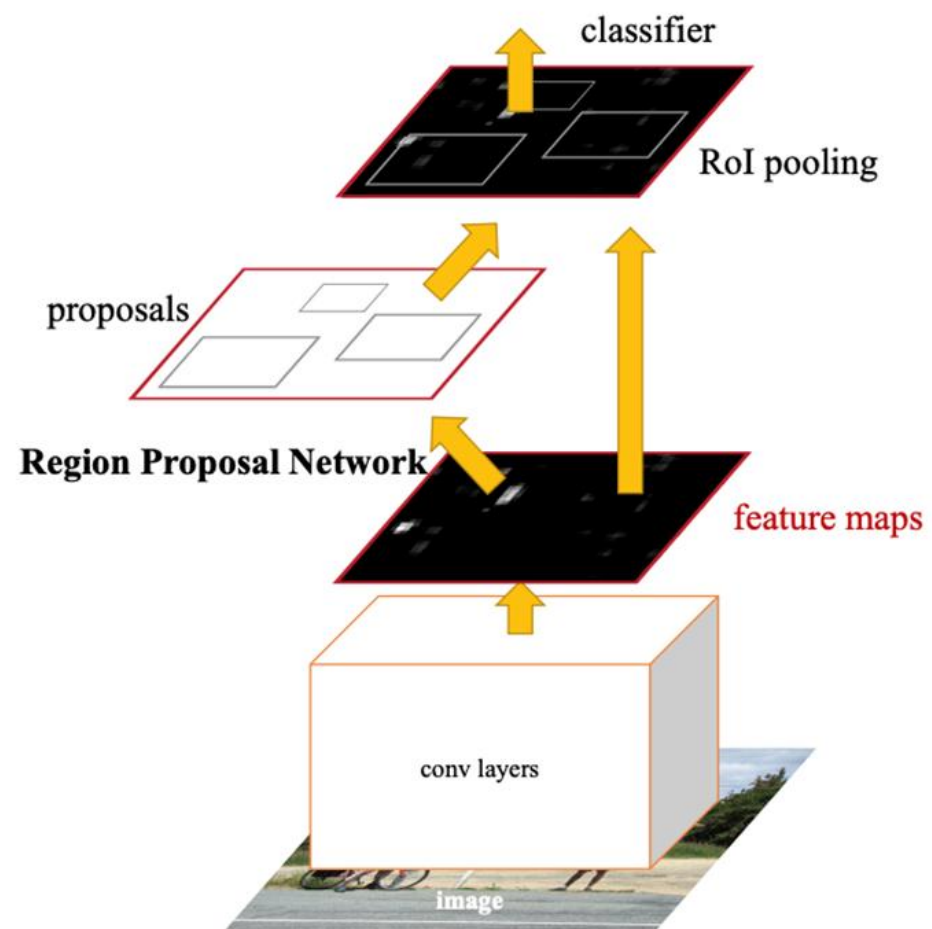
- 탐지 정확도 높음 (특히 작은 물체)
- 고정밀 탐지가 필요한 작업에 적합

◆ 단점

- 속도가 느림 (실시간 x)
- 구조가 복잡함

- Step1 : Feature Extraction : Image를 CNN(ResNet, VGG)으로 통과시켜 feature map 생성
- Step2 : RPN(Regional Proposal Network). 슬라이딩 윈도우 방식으로 feature map위에서 Anchor Box를 탐색. 객체존재여부와 Bbox 위치 조정
- Step3 : RoI Pooling : RPN에서 뽑은 다양한 크기의 ROI들을 고정된 크기로 변환
- Step4 : Classifier + Regressor. 각 ROI를 fully Connected Layer에 넣어 Class 분류 및 박스 Refinement(정확한 위치 조정)





▶ “복잡한 두 단계를 하나로 합쳐서, 이미지를 한 번에 보고 물체가 뭔지 즉시 예측하자!”

YOLO 작동 방식

- 이미지를 $S \times S$ 그리드로 분할
- 각 셀에 :
 - 물체가 있는지 확인
 - 물체의 종류(Class)와 위치(Bounding Box) 예측

◆ 장점

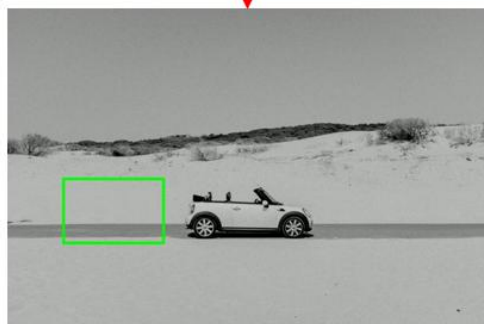
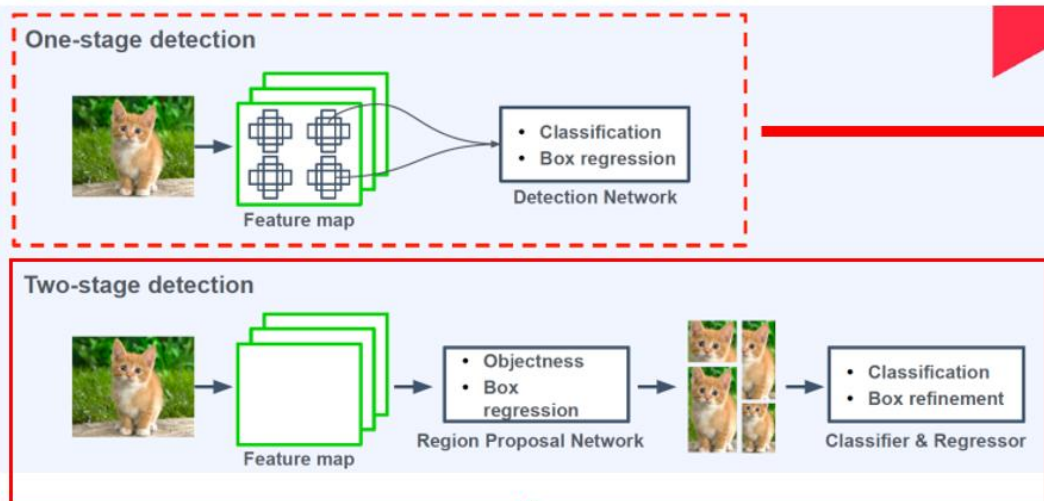
- 실시간 처리 가능 (30~60fps)
- 구조 단순, 빠르고 가볍다
- 드론, CCTV, 로봇 등에 적합

◆ 단점

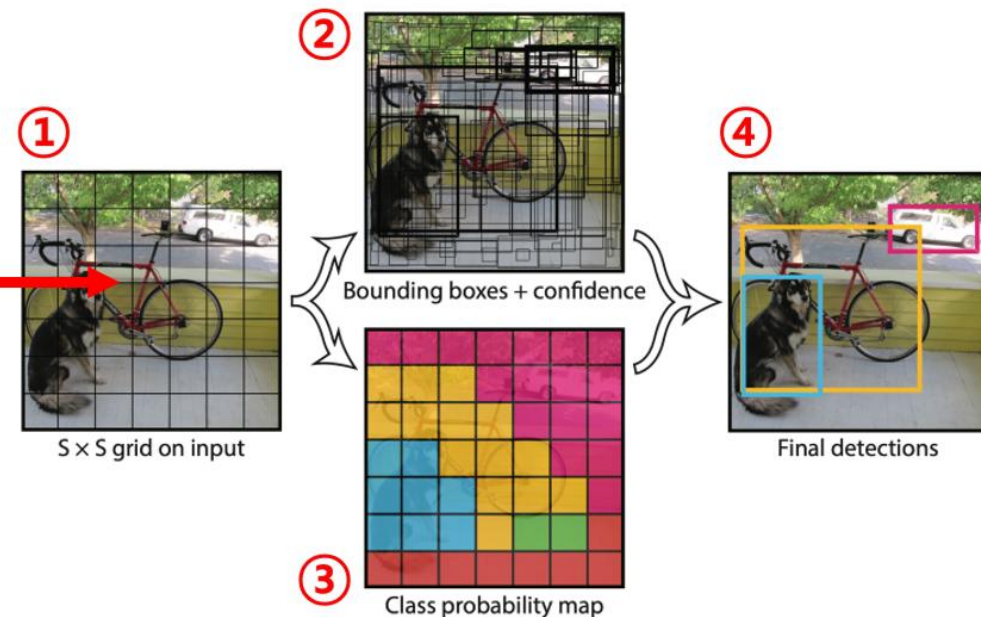
- 처음에는 작은 물체 인식이 어려웠지만 최신 버전에선 개선됨

▶ YOLO(You Only Look Once)

- 이미지 전체 한번에 넣어서 detection 진행
- One-Stage detection을 통해 인식률은 낮지만 FPS 높아 상용화에 이점
- 실시간 처리가 가능해 자율 주행, 의료 영상 분석에 많이 활용

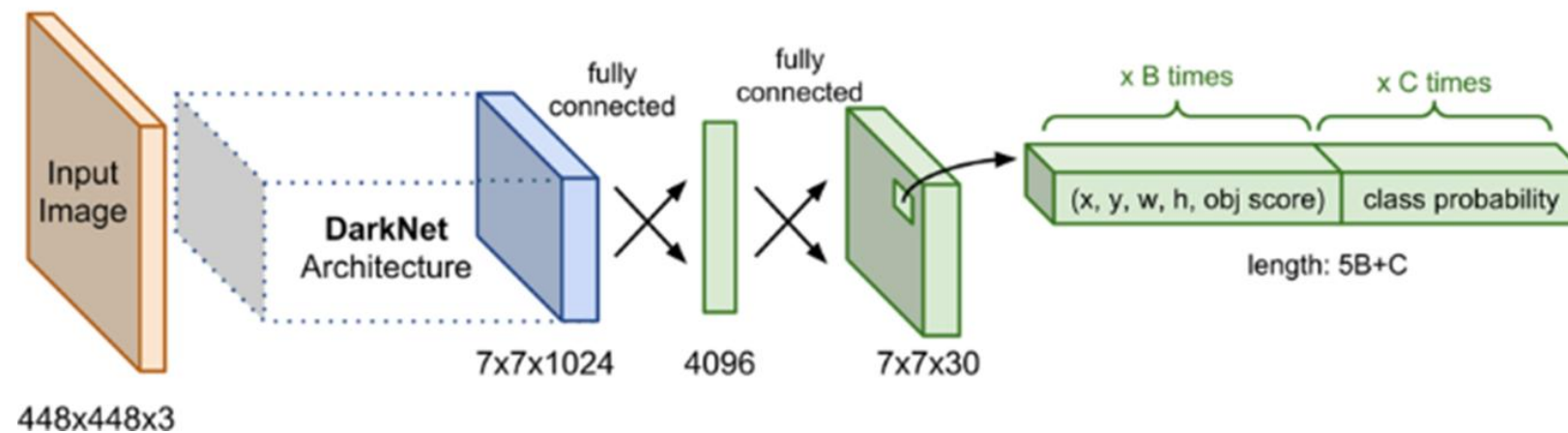


- ▶ 기존 객체 인식 방식
▶ 다양한 크기의 객체 인지가 가능하지만 실시간 처리에 문제가 있음
→ 이미지 크기가 클 경우 연산 부하가 증가

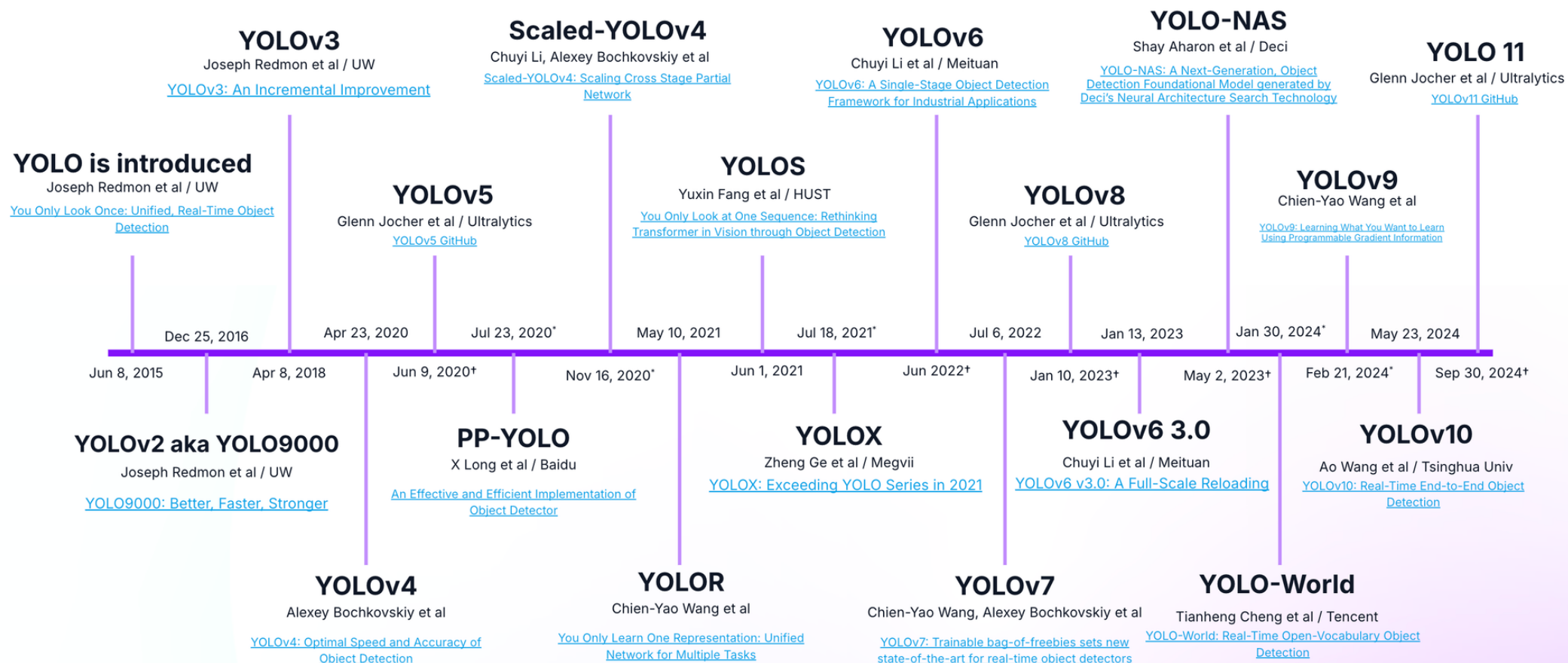


- ① 원본 이미지를 동일한 크기의 그리드(grid)로 나눔
- ② 객체의 바운딩 박스 처리와 개수를 예측해 신뢰도를 계산
- ③ 사전에 정의된 N개의 클래스에 대한 확률을 예측해 계산
- ④ 예측한 바운딩 박스(굵은 영역)와 클래스 확률을 결합해 객체 인식

- ▶ “복잡한 두 단계를 하나로 합쳐서, 이미지를 한 번에 보고 물체가 뭔지 즉시 예측하자!”
- x, y, w, h : 바운딩 박스 정보
 - obj score : 해당 바운딩 박스의 점수(객체가 존재하는지에 대한 확률)
 - Class probability : 해당 객체가 어떤 class에 속할지



YOLO timeline

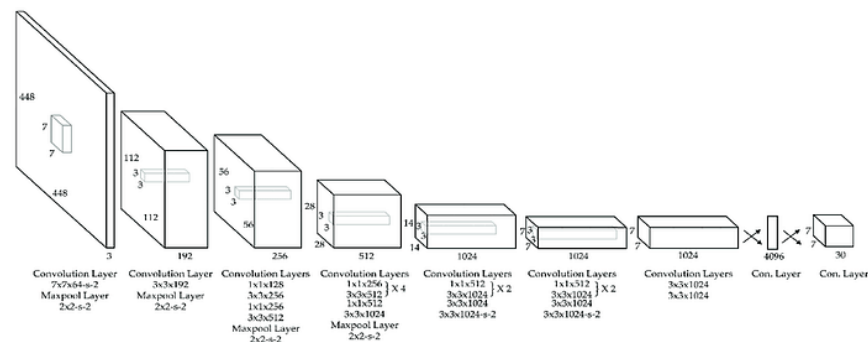
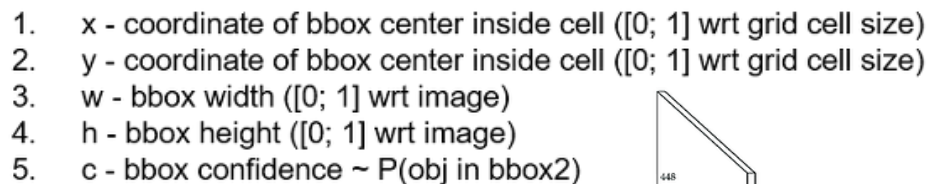
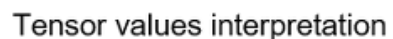


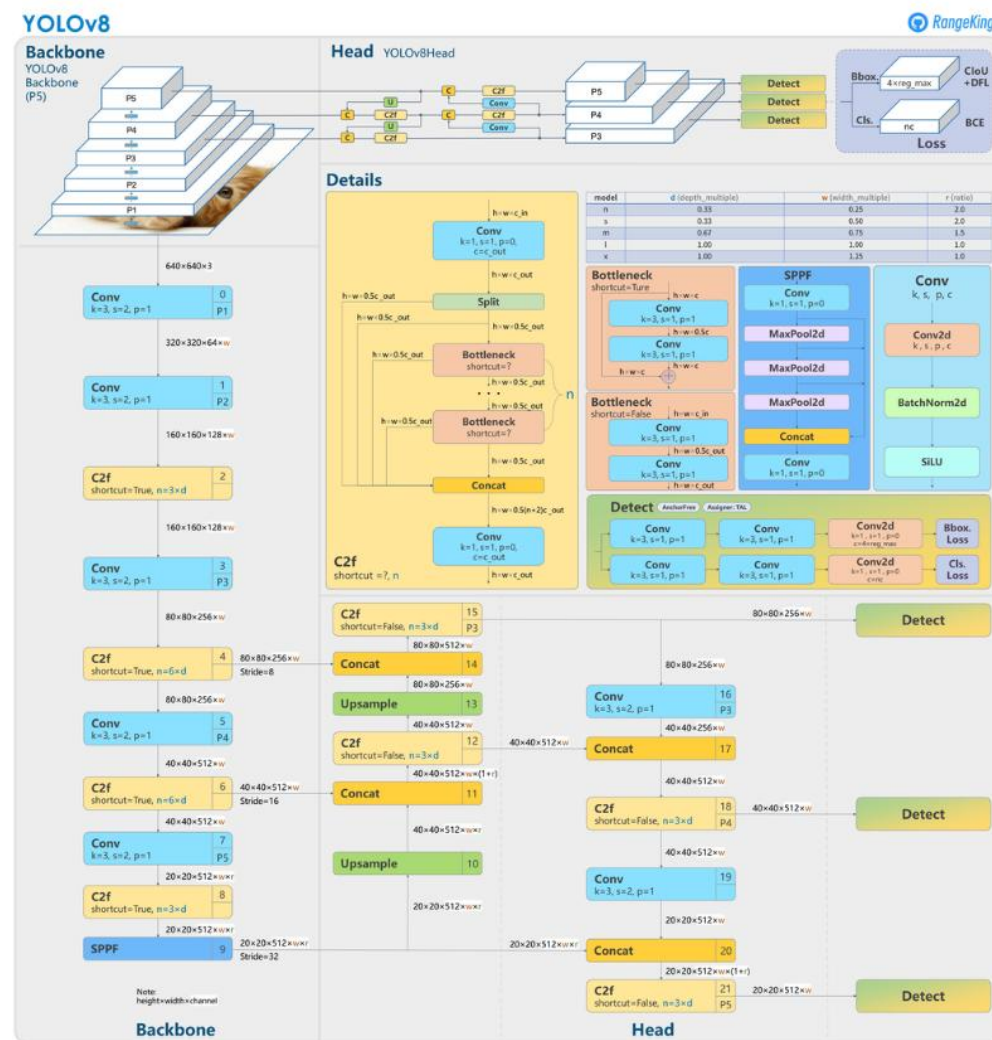
^{*} Denotes paper updated after first publication date

⁺ Denotes repository predates paper publication date



버전	특징	한 줄 요약
v1 (2016)	논문 첫 등장, 빠르지만 정확도 낮음	속도 혁신의 시작
v2 (YOLO9000)	다중 클래스 인식 가능, COCO + ImageNet	9000개 클래스도 가능!
v3	작은 객체 성능 향상, 다중 스케일	실제 실용성 확보
v4	성능 최적화, 훈련 속도 개선	빠르고 정확하게!
v5 (Ultralytics)	공식 논문 x, 사용성 최고 (PyTorch)	가장 대중적인 실습 버전
v6	경량화, 모바일 최적화	실시간 모바일 대응
v7	속도+정확도 밸런스 향상	리얼타임 최고 성능
v8 (2023)	클래스 없는 탐지(Class-agnostic), Segment 지원, CLI 강화	가장 똑똑하고 유연함



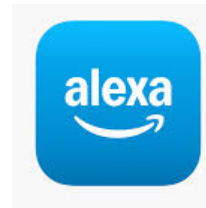


항목	Faster R-CNN	YOLO
구조	2단계 (제안 → 분류)	1단계 (직접 예측)
속도	느림	빠름 (실시간)
정확도	높음 (작은 물체도 잘 잡음)	최신 버전에서는 거의 유사
사용 용도	정밀 탐지 (예: 의료 영상)	실시간 처리 (로봇, 드론)
실습 난이도	높음	낮음 (YOLOv5, v8 실습 쉬움)

YOLOv5	YOLOv8	YOLOv11
2020년 출시	2023년 출시	2024년 출시
PyTorch 기반으로 구현	YOLOv5을 바탕으로 완전 새롭게 설계된 아키텍처 다양한 컴퓨터 비전 지원	YOLOv8 아키텍처기반으로 더욱 최적화된 구조와 효율성을 제공 실시간 애플리케이션에 적합
Object Detection	Object Detection, Classification, Segmentation, Tracking	Object Detection, Classification, Segmentation, Tracking
추론 속도 빠름	추론 속도 빠름	추론 속도 가장 빠름
정확도(mAP) 우수	정확도 향상됨	정확도 최고수준
자세추정 안됨	제한적 자세추정 가능	자세추정
고정밀 산업용 검사에 제한적	고정밀 산업용 검사	고정밀 산업용 검사
학습자료가 풍부하고 입문자나 학습용으로 적합	다양한 작업 지원, 높은 정확도와 효율성 제공 연구나 개발에 적합	실시간 응용 분야나 리소스가 제한된 환경에서도 뛰어난 성능 발휘

▶ Wakeup Word

- 본격적인 명령 수행이나 대화 입력을 본격적으로 처리하기 앞서 기기의 시동을 걸기 위한 문구
- Apple : 시리야~
- Google : OK Google
- Amazon : Alexa

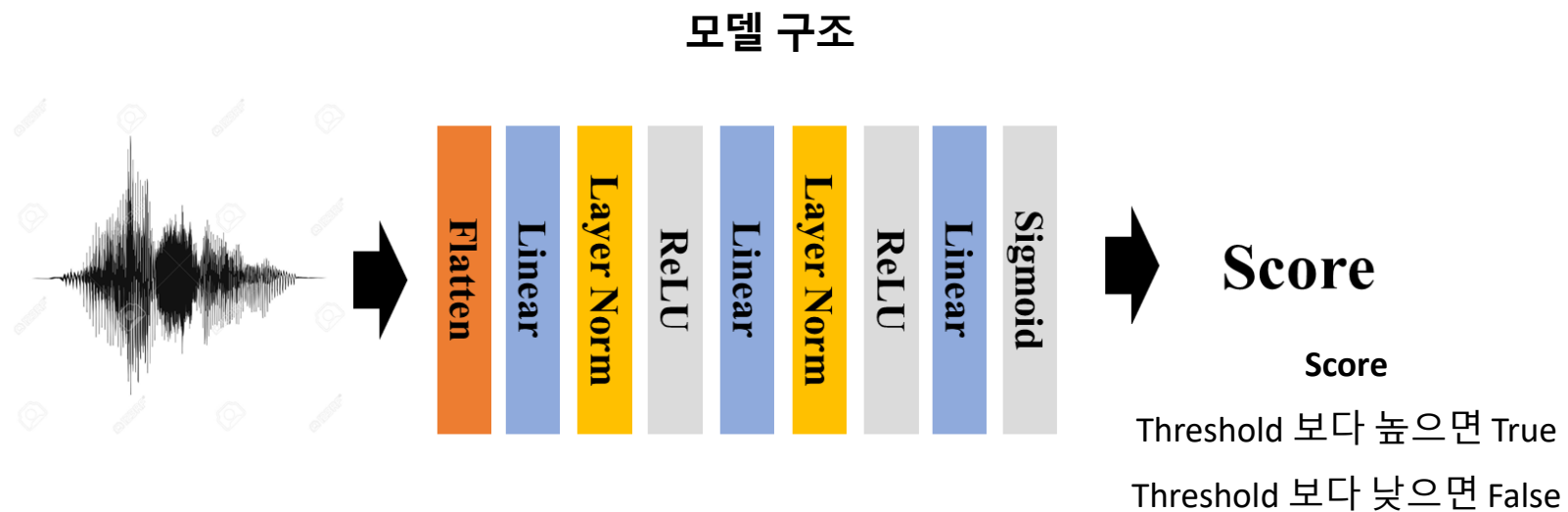
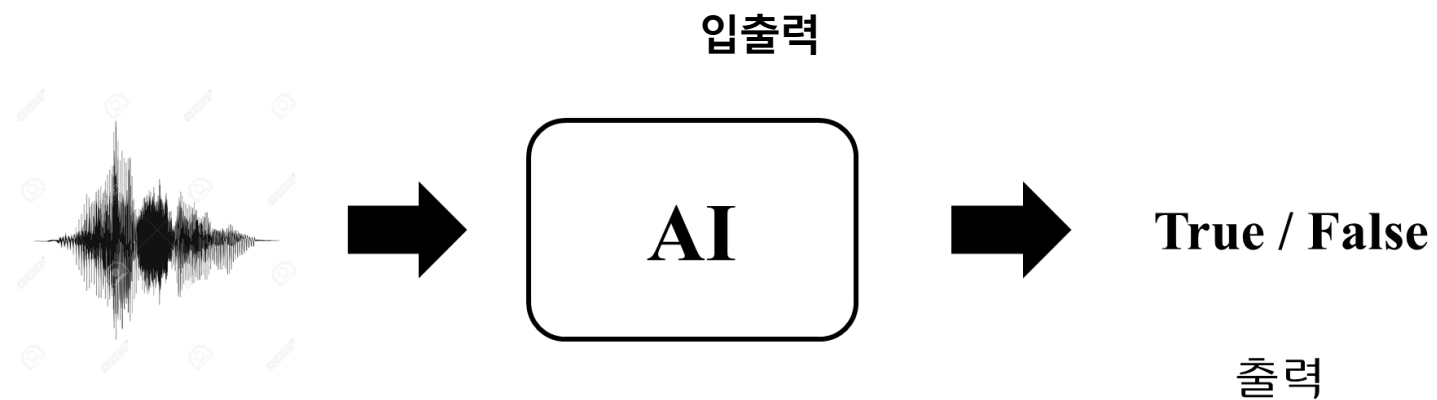


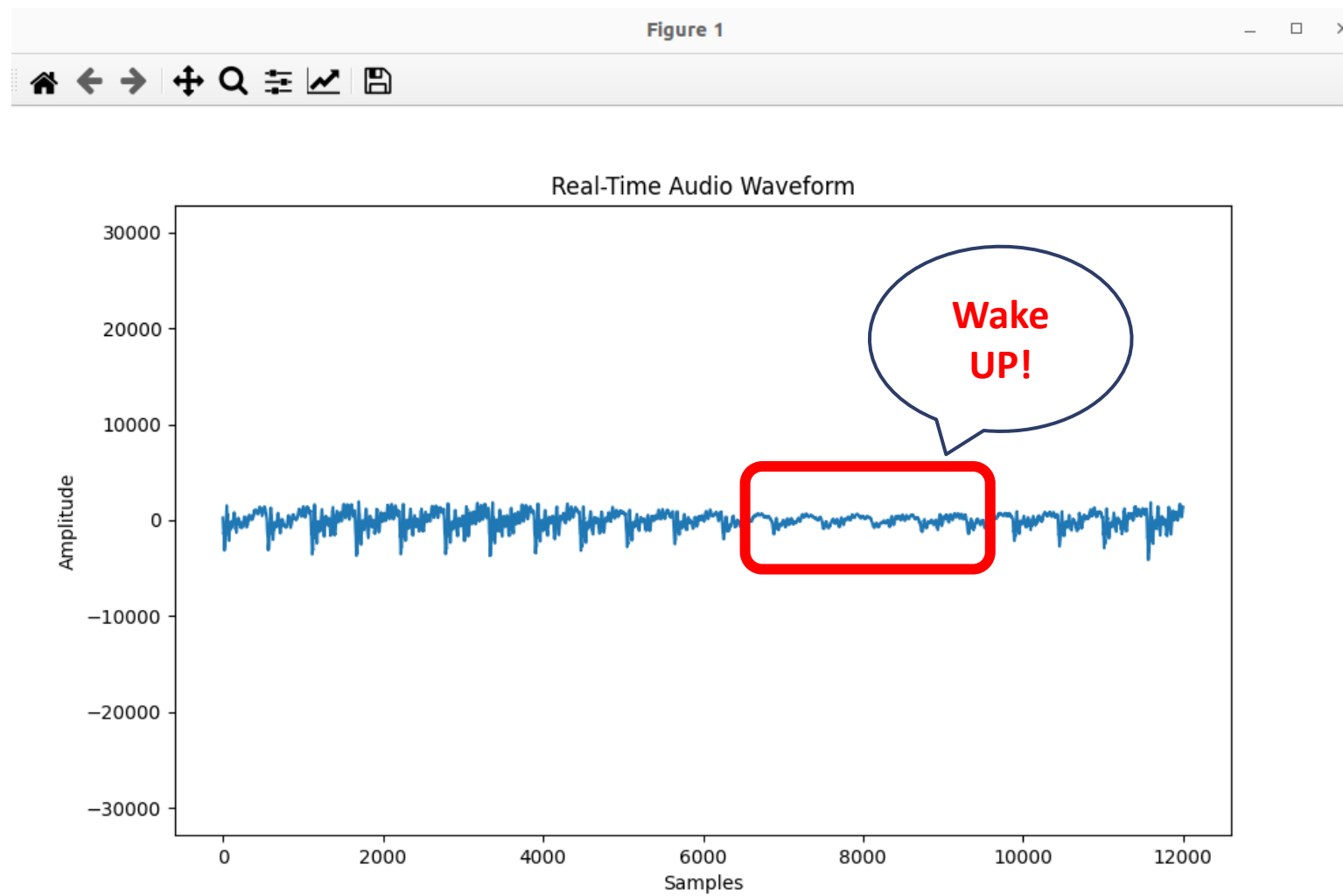
▶ Wakeup Word Model

- Wakeup Word가 말로 입력되었는지를 빠르고 정확하게 감지하는 데 특화된 머신러닝 모델

- Wake Word : 특정 단어를 말하면 AI가 그 이후 음성을 인식하기 시작하는 트리거 역할을 하는 단어
- 사용목적 : 계속 STT를 실행해두면 Resource 낭비이므로 필요할 때만 인식(사용자 편의성 향상 + 성능 효율성 유지)
- Wake Word : 한 단어만 기다림
- STT : 모든 말을 듣고 받아 적어야 함

구성 요소	설명
마이크 입력	실시간 오디오 스트림 순서
특징 추출	오디오 → 특징 벡터로 변환(보통은 mel-spectrogram등 사용)
모델 추론	특정 wakeup word일 확률 출력
임계값 판단	Confidence(신뢰도) > threshold 이면 '감지됨'으로 처리





항목	Wake Word	STT(Speech To Text)
정의	특정 단어나 구문("Hey Siri" 등)을 실시간 감지 하는 기술	사용자의 음성을 텍스트로 변환 하는 기술
목적	시스템을 깨우거나 명령 대기 상태로 만들기 위해 사용	사용자의 음성 내용을 이해하거나 처리 하기 위해 사용
역할	트리거 역할 (시작 신호)	정보 전달 (실제 내용 인식)
기술	Porcupine, Precise	Whisper, Vosk, Google STT
데이터	특정 키워드 예시 수백~수천 개	일반 대화, 문장 전체 데이터 수천 시간
학습 대상	특정 단어 발화 여부	모든 단어, 문장 의미 인식
작동시점	항상 백그라운드에서 감시 중	Wake word 후 또는 명시적 트리거 후 작동
입력길이	매우 짧음 (보통 한 단어 또는 짧은 구)	전체 문장, 길게 말하는 내용 모두 포함
처리범위	특정 키워드만 감지	전체 발화 내용 분석 및 변환
트리거 방식	항상 청취하다가 특정 단어에 반응	트리거 후 음성 전체 인식
활용방식	"Hey Google", "Alexa", "Hey BMW", "Hello Rokey" → 기동	"오늘 날씨 알려줘", "집으로 가는 길 알려줘", "앞으로 가" → 동작 수행

라이브러리	장점	단점
Porcupine	- 초경량, 빠른 속도(온디바이스 추론) - 다양한 플랫폼 지원(Python, Android, iOS등) - 내장 Wake word 사용 또는 사용자 정의 모델 생성 가능	- 사용자 정의 모델은 회원 가입 및 웹에서 생성 필요 - 무료 요금제엔 제약 있음
Mycroft Precise	- 오픈 소스 - 사용자 정의 Wake word 모델 훈련 가능(텐서플로우 기반)	- 초기설정, 훈련, 마이크 연결 등 환경 구성 복잡 - 초보자에게 다소 난이도 있음
Vosk	- 경량 STT엔진 - 전체 문장을 실시간으로 받아 특정 키워드 필터링 가능 - 오프라인 사용 가능	Wake word전용은 아님(STT 기반 키워드 탐지로 유사 기능 가능)
Whisper	- 고정확도(음성 → Text) - 딥러닝 기반 강력한 STT모델 - 멀티언어지원	리소스 요구가 큼

PICOVOICE

<https://picovoice.ai/>

Hey Mycroft

<https://github.com/MycroftAI/mycroft-precise>

αcephei

<https://alphacephei.com/vosk/>OpenAI
Whisper<https://openai.com/index/whisper/>

우리가 사용할 Wake up Word 라이브러리

- 오픈소스 Wake Word Detection 라이브러리
- TFLite(TensorFlow Lite) 기반 모델 사용
- PyTorch로 학습된 모델을 .tflite로 변환하여 사용
- 다양한 커스텀 모델 사용 가능(hello rokey, hey mycobot, alexa 등)

OpenAI Wake Word 장단점

장점	설명
오픈소스 & 무료	누구나 설치하고 바로 실습 가능. 연구, 교육, 취미 모두 OK
로컬 실행 가능(오프라인 지원)	인터넷 없이도 Wake Word 탐지 가능 → 보안성, 속도 우수
경량 TFLite 모델 사용	실시간 추론 가능, Raspberry PI 등 저사양 환경에서도 사용 가능
커스텀 Wake Word 모델 지원	직접 만든 Wakeword 모델(.tflite) 로드 가능
간단한 사용법	Model, Predict, Threshold등으로 직관적인 API구성
다양한 샘플 모델 제공	"Hello World", "Alexa", "Hey Robot"등 샘플 모델 내장
단점	설명
사용자 모델 훈련은 직접해야	커스텀 모델 훈련도구는 따로 제공하지 않음(직접 학습 필요)
잡음/환경 변화에 민감할 수 있음	마이크 품질, 음량, 잡음 등 실환경에서 성능차이가 큼
실험적 기능이 포함	아직 대규모 커뮤니티는 없고, 문서도 상대적으로 간단함
다국어 지원이 명확하진 않음	기본 모델은 주로 영어 기반
Threshold 조절이 필요	Confidence값 튜닝없이 바로 쓰면 오탐/누락 가능성 있음

STT(Speech-to-Text)

사람의 음성을 받아서 → 컴퓨터가 텍스트로 변환하는 기술

- 오픈소스 Wake Word Detection 라이브러리
- TFLite(TensorFlow Lite) 기반 모델 사용
- PyTorch로 학습된 모델을 .tflite로 변환하여 사용
- 다양한 커스텀 모델 사용 가능(hello rokey, hey mycobot, alexa 등)

사용 예시

- 스마트폰 음성 입력
- 인공지능 스피커 (네이버 클로바, 카카오미니 등)
- 콜센터 자동 상담 시스템
- 자막 자동 생성
- 음성 명령을 통한 로봇 제어



OpenAI의 STT API 이용

항목	설명
API 기반	음성 데이터를 OpenAI 서버에 전송해서 → 결과 텍스트를 받음
녹음	MicController로 마이크에서 음성 녹음
변환 과정	녹음된 음성을 Base64로 인코딩해서 전송
모델	gpt-4o-audio-preview 모델을 사용
결과	모델이 음성 내용을 정확히 텍스트로 바꿔서 반환

그외 STT(Speech To Text)

이름	방식	한국어지원	오프라인	특징
Google Speech To Text	클라우드 API	좋음	X	고정밀, 실시간, 무료, 일정량 제공
Whisper(OpenAI)	오픈소스(로컬)	아주 좋음	O	높은 정확도, GPT기반, 다양한 언어
Vosk	오픈소스(로컬)	가능	O	경량화 모델, Raspberry Pi 지원
Naver CLOVA Speech	클라우드API	아주 좋음	X	한국어 특화, API 사용
Kakao STT	클라우드API	좋음	X	한국어 적당히 지원, REST API
Azure Speech Service	클라우드API	좋음	X	글로벌 서비스, 다양한 설정
DeepSpeech(by Mozilla)	오픈소스	약함	가능	개발 중단, 학습 필요
AssemblyAI/RevAI	클라우드API	영어 특화	X	영어 정확도 높음

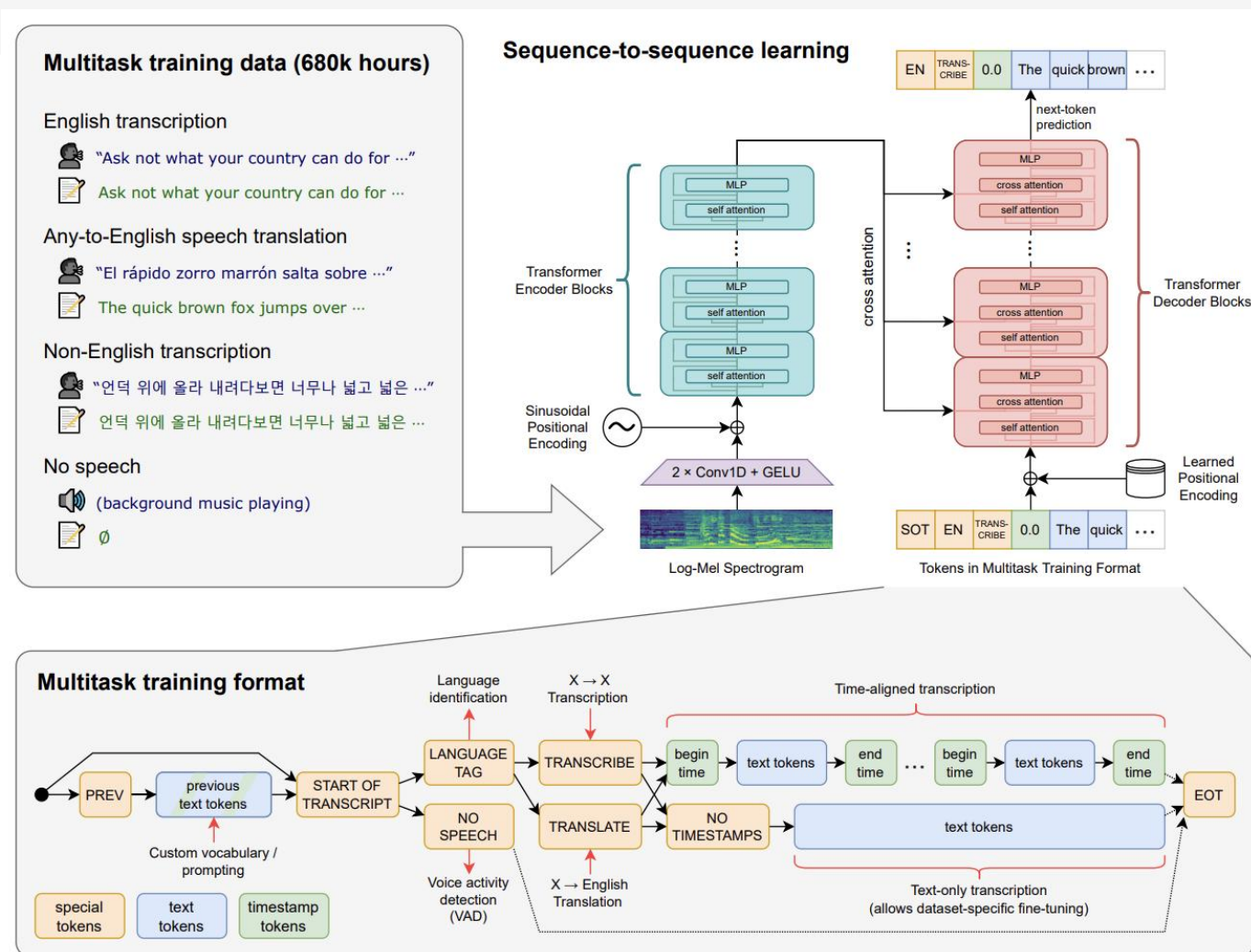
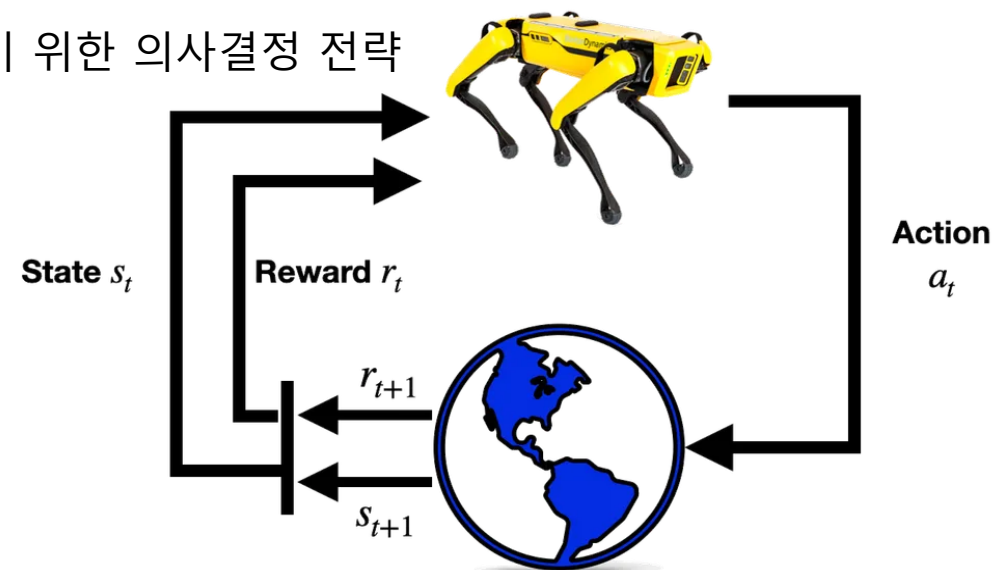


Figure 1. Overview of our approach. A sequence-to-sequence Transformer model is trained on many different speech processing tasks, including multilingual speech recognition, speech translation, spoken language identification, and voice activity detection. All of these tasks are jointly represented as a sequence of tokens to be predicted by the decoder, allowing for a single model to replace many different stages of a traditional speech processing pipeline. The multitask training format uses a set of special tokens that serve as task specifiers or classification targets, as further explained in Section 2.3.

강화 학습의 구성요소

- a. 에이전트(Agent) : 강화 학습 상황에서 행동의 주체가 되는 시스템
- b. 환경(Environment) : 에이전트가 상호작용하는 외부 세계
- c. 상태(State) : 에이전트가 환경을 관찰한 결과
- d. 액션(Action) : 환경을 관찰한 에이전트의 선택(행동)
- e. 보상(Reward) : 에이전트가 현재 주어진 상태에서 액션을 선택할 때의 결과값
- f. 정책(Policy) : 에이전트가 학습하는 과정에서 발전하도록 하기 위한 의사결정 전략



강화 학습의 입력과 출력

- 상태(State)를 입력받아 행동(Action)을 출력하는 정책을 학습
- 이에 따라 성공적인 RL 적용을 위해서는 입력/출력(I/O)가 중요함

RL 요소	역할
입력 : 상태(State)	환경의 현재 상황을 나타내는 관측 벡터 로봇이 판단하는데 필요한 모든 정보
출력 : 행동(Action)	환경에 영향을 미치는 명령 또는 제어 신호 로봇이 수행해야 할 동작(관절 각도, 그리퍼 명령 등)
보상 (Reward)	행동을 취했을 때 환경으로부터 받는 피드백 (학습 목표)

1. DQN과 PPO의 정의 및 특징

강화 학습(RL)은 에이전트가 환경과의 상호작용을 통해 **최적의 행동 정책**을 학습

DQN (Deep Q-Network)

- 심층 신경망을 사용하여 Q-함수를 근사하는 가치 기반 알고리즘
- 특징: 유한하고 셀 수 있는 행동, 복잡한 행동 공간을 분해하여 활용도 높임
- 프로젝트 활용 : 물품의 방향/위치 선택 등 명확히 분리된 선택의 과제

PPO (Proximal Policy Optimization)

- 정책 경사 계열로 정책 파이를 직접 학습하는 알고리즘, 안정성 높아 가장 널리 사용
- 특징: 연속적 행동 공간(실수 값으로 표현되는 행동) 동작 제어나 안정적인 정책 업데이트가 필요할 때 이상적
- 프로젝트 활용 : 로봇 관절의 정밀한 궤적 제어

Deep Q-learning Network 입출력

항목	내용
적합 행동	이산적 행동, 회전 인덱스, 배치 위치 등
입력 : 상태(S)	그리드 점유 정보, 현재 조각 ID, 조립 진행률 등 로봇의 카메라/센서 데이터를 기반으로 정규화하여 벡터로 구성
출력 : 행동(A)	행동 인덱스 $\rightarrow$ (방향 인덱스, 위치 인덱스) 쌍 계층적 DQN을 통해 방향(Q_{ori})과 위치(Q_{pos})를 분리하여 예측한 후 합산
활용 아이디어	파지 전략 선택: 물체를 집을 때 가장 안정적인 그리퍼 자세를 선택

Deep Q-learning Network

적용분야	활용된 DQN 기법	핵심 역할
행동 결정	계층적 DQN	거대한 행동 공간을 방향(Qori) 과 위치(Qpos) 예측으로 분해하여 학습 효율 높임
학습환경	로봇 친화적 환경 설계	수직 경로 확보 여부, 물리적 지지 안정성 등 로봇의 실제 제약을 환경 필터에 통합 실행 불가능한 행동 학습하지 않도록 방지
보상설계	보상 셰이핑	성공/실패 외에 우선 배치 보너스(+30), 응집 보너스(+2) 등을 부여해서 구조적으로 안정되고 효율적인 조립 순서 학습 유도
실행안전	XYZ 특이점 가드	DQN이 결정한 회전 자세가 로봇의 관절 특이점(90도 초과)을 유발할 경우, 자세를 안전 각도(90도 이하)로 보정하고 위험한 자세는 여러 안전 회전으로 분할 실행

Proximal Policy Optimization 입출력

항목	내용
적합 행동	연속적 또는 이산적 행동 정책 업데이트의 안정성이 높음 관절 궤적 제어
입력: 상태(S)	게임 상태, 상대 관측 정보, 승률 추정치 등
출력: 행동(A)	PPO 정책으로 전략적 의사 결정(여러 행동 중 하나)을 내림
활용 아이디어	유연한 동작 제어 : 물체를 놓을 때 Gripping Force나 최종 TCP 높이를 학습하여 유연 조립/파지 실패율 개선

Proximal Policy Optimization

적용분야	활용된 PPO 정책 결과	활용 예시
의사결정	정책 추론	현재 게임 상태와 비전/음성 센서로 관측한 상대방 정보를 기반으로 최적의 행동 추론
센서통합	관측 벡터	카메라로 획득한 정보를 정규화하여 정책 입력으로 사용 이를 통해 로봇의 인지 능력을 추후 행동에 직접 반영
물리적행동	Action Server	추론된 행동은 다른 Server를 통해 다음 로봇 동작으로 변환되어 실행

강화학습 활용 방안 1

활용 분야	적용할 RL 기법 및 목표	연계 방안
복잡한 이산 행동	계층적 DQN (행동 분해)	비정형 물체에 대한 '파지 전략(그리핑 위치/방향)'을 학습하여 안정성 개선
동작 계획/안정성	환경 엔지니어링 및 보상 셰이핑	Pick&Place시 충돌 없는 경로를 학습하도록 경로를 보상에 추가하거나, 자세 안정성을 상태 필터로 적용
실시간 센서 통합	PPO 정책 추론 및 상태 설계	음성/비전 정보(물체 종류, 거리, 작업자 위치)를 상황에 맞는 동작 속도나 안전 거리를 결정하도록 학습
정밀 제어	PPO (연속 행동)	그리퍼의 Gripping Force나 Pick Height 등 연속적인 물리량을 RL이 결정하도록 하여, 힘 제어 성능 개선

강화학습 활용 방안 2

활용 분야	활용 아이디어
복잡한 의사결정	DQN(계층적): 조립 순서, 목표 위치 등 전략적 단계에서 선택의 폭을 좁히는 메타 정책으로 활용
정밀 조작	PPO: 그리퍼의 힘 제어 또는 미세한 궤적 수정에 PPO를 적용하여 정밀도를 정량화 및 최적화
센서 통합	DQN/PPO(상태 정의): Depth Camera의 Pointcloud 정보, STT Keyword 등을 상태벡터에 추가하여, 로봇이 인지적 피드백을 기반으로 행동을 결정하도록 설계
안전성 확보	환경 제약: DQN/PPO 학습 환경에 로봇 관절 한계나 충돌 예상 경로를 강력한 페널티 또는 행동 마스킹으로 포함하여 학습 시 안정성 확보

ROKEY BOOT CAMP

수고하셨습니다.